



**Hochschule
Hof**

University of
Applied Sciences

Software Engineering for Industrial Applications
Mandatory Internship SoSe-2026

**Development of
AI-powered Retrieval-Augmented
Generation (RAG) system
“Software Specification and Design Report”**

Submitted To

Prof. Dr. Matthias Meitner
Hof University of Applied Sciences

Submitted By

Md Fahim Hossain
Student ID: 00154924

Table of Contents

1. Introduction & Company Overview	3
1.1. Introduction	3
1.2. Company Overview	3
1.3. Department Overview	4
1.4. Motivation for the Project.....	4
2. Role & Tasks	4
3. Project Overview	5
4. Tools & Frameworks.....	6
4.1. Backend Development	7
4.2. Artificial Intelligence and Document Processing	7
4.3. Frontend Development	7
4.4. Development and Deployment Tools	8
5. Specification	8
5.1. Purpose, Scope, Users, and Constraints	8
5.2. Functional Requirements	9
5.3. Non-functional Requirements	10
5.4. User Stories	10
5.5. Use Case Diagrams and Use Case Descriptions	13
5.6. External Interfaces	18
5.7. User Interface Mockups	21
5.7.1. Login Interface.....	21
5.7.2. User Dashboard.....	22
5.7.3. Document Upload Interface	22
5.7.4. Document Details and Processing Status.....	23
5.7.5. Conversation Interface	24
5.7.6. Conversation History	24
5.7.7. Administrative Dashboard.....	25
6. Design	26
6.1. System Architecture	26
6.2. Component Diagram	27
6.3. Class Diagram.....	29

6.4.	Sequence Diagrams	31
6.5.	Activity Diagrams	33
6.6.	Database Design.....	35
7.	Conclusion	37
8.	References	38

List of Tables

Table 1:	Technologies Used in the Development of RAG System	7
Table 2:	Relationships Between the Primary Database Entities	37

List of Figures

Figure 1:	Overall Use Case Diagram of the AI-powered Document Retrieval System	13
Figure 2:	PostgreSQL Database Interface	19
Figure 3:	Embedding Model Interface Used for Document Vector Generation	19
Figure 4:	Language Model Interface Used for Context-Aware Response Generation	20
Figure 5:	Background Job Processing Interface	20
Figure 6:	Docker Container Environment	21
Figure 7:	Login Interface	22
Figure 8:	User Dashboard	22
Figure 9:	Document Upload Interface	23
Figure 10:	Document Details and Processing Status Interface	23
Figure 11:	Conversation Interface	24
Figure 12:	Conversation History Interface	25
Figure 13:	Administrative Dashboard	25
Figure 14:	High-Level Architecture of the AI-Powered Enterprise RAG System	27
Figure 15:	UML Component Diagram of the AI-Powered Enterprise RAG System	29
Figure 16:	UML Class Diagram of the AI-Powered Enterprise RAG System	30
Figure 17:	Sequence Diagram for Document Upload and Processing Workflow	31
Figure 18:	Sequence Diagram for AI-Assisted Question Answering Workflow	32
Figure 19:	Activity Diagram for Document Upload and Processing.....	33
Figure 20:	Activity Diagram for AI-Assisted Question Answering Workflow	34
Figure 21:	Entity Relationship Diagram of the AI-Powered Document Retrieval System	36

1. Introduction & Company Overview

1.1. Introduction

As part of the Master's program in Software Engineering for Industrial Applications at Hof University of Applied Sciences, students are required to apply theoretical knowledge in a practical industrial environment through project-based work. This report documents the software specification and design activities performed during the development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document management and knowledge retrieval.

Modern organizations generate and maintain large volumes of technical documents, reports, manuals, process descriptions, and operational guidelines. Accessing relevant information from these documents often requires significant manual effort, leading to reduced efficiency and increased time spent searching for knowledge. To address this challenge, an intelligent document retrieval system was designed and developed that combines document processing, semantic search, and Artificial Intelligence technologies to provide context-aware answers based on organizational documents.

The project was developed during my work as a Working Student at Siemens Energy. The development process involved requirements analysis, software specification, system design, architecture planning, implementation, and validation of the proposed solution. This report focuses on the specification and design aspects of the system, including functional and non-functional requirements, use cases, system architecture, interface design, and UML models.

The objective of the project was to develop a scalable and maintainable software solution that can process organizational documents, retrieve relevant information, and help users find answers through natural language queries.

1.2. Company Overview

Siemens Energy is one of the world's leading energy technology companies, operating across the entire energy value chain. The company develops innovative solutions for power generation, transmission, storage, and industrial applications while supporting the global transition toward sustainable and reliable energy systems.

With operations in more than 90 countries and a workforce of approximately 100,000 employees worldwide, Siemens Energy serves customers in both conventional and renewable energy sectors. The company provides technologies and services for gas turbines, steam turbines, power generation systems, grid infrastructure, hydrogen solutions, industrial applications, and digital transformation initiatives.

Digitalization plays a critical role in Siemens Energy's strategy. The company continuously invests in advanced software solutions, data-driven decision making, artificial intelligence, cloud technologies, and process automation to improve operational efficiency and support business innovation. These initiatives enable employees and stakeholders to access information more effectively, optimize workflows, and enhance organizational productivity.

1.3. Department Overview

The project was conducted within the SE GT DG OE department of Siemens Energy.

The department is involved in supporting digitalization initiatives, process optimization, and data-driven solutions that enhance operational efficiency across business functions. Various software tools, reporting systems, and analytical platforms are utilized to manage information, automate processes, and improve collaboration among stakeholders.

As part of ongoing digital transformation efforts, the department actively explores modern technologies such as cloud platforms, data analytics, artificial intelligence, and enterprise software systems. These technologies are used to streamline business processes, improve knowledge management, and support informed decision-making.

The development of the AI-powered document retrieval system aligns with these objectives by providing an intelligent mechanism for managing and accessing organizational knowledge. The project contributes to the broader goal of improving information accessibility and reducing the effort required to locate relevant content within large collections of enterprise documents.

1.4. Motivation for the Project

Organizations frequently store critical knowledge across numerous documents, including technical manuals, operational procedures, project documentation, and internal reports. Although these documents contain valuable information, retrieving specific content often requires manual searching, which can be time-consuming and inefficient.

The increasing adoption of Artificial Intelligence technologies has created opportunities to enhance document management systems through semantic understanding and context-aware information retrieval. Instead of relying solely on keyword-based searches, modern AI-assisted systems can understand user intent and identify relevant information based on contextual meaning.

The motivation behind this project was to design and develop a software solution capable of improving access to organizational knowledge while maintaining a user-friendly experience. By integrating document processing, semantic retrieval mechanisms, and AI-assisted response generation, the proposed system aims to support employees in obtaining accurate information quickly and efficiently.

The project also provided an opportunity to apply software engineering principles, including requirements engineering, system design, architectural modeling, and software development methodologies within a real industrial environment.

2. Role & Tasks

During my internship at Siemens Energy, I worked as a Working Student in the Digital Solutions team, contributing to both enterprise software development and data engineering activities. My responsibilities included supporting internal data analytics initiatives while leading the design and development of an AI-powered Retrieval-Augmented Generation (RAG) system for enterprise document

management and knowledge retrieval. Throughout the internship, I collaborated with software engineers and business stakeholders to analyse requirements, design software components, implement new features, and document the developed solution.

My primary responsibility was the development of the RAG system. I participated in the complete software development lifecycle, including requirements analysis, system design, backend development, user interface implementation, and technical documentation. The system was designed to provide secure and efficient access to enterprise knowledge through semantic search and AI-assisted question answering.

As part of this work, I developed the document management and processing workflow, enabling users to upload documents, monitor processing progress, and retrieve information through a conversational interface. I also implemented semantic retrieval, conversation management, and background processing features to support efficient document indexing and question answering. In addition, I contributed to improving the system architecture, optimizing database operations, and preparing the application for containerized deployment.

Alongside the RAG project, I supported enterprise data engineering activities involving Python, SQL, and Snowflake. My work included developing and validating data processing workflows, transforming business data for reporting purposes, and contributing to the development of Power BI dashboards in collaboration with domain experts.

Throughout the internship, I actively participated in team discussions, code reviews, and technical knowledge-sharing sessions. These activities improved both the software and my understanding of collaborative development practices. I also prepared technical documentation to support future maintenance and continued development of the application.

The combination of enterprise software development, AI-assisted document retrieval, and data engineering provided valuable experience in designing scalable software systems, collaborating within multidisciplinary teams, and applying software engineering principles to solve practical industrial challenges.

3. Project Overview

The project developed during this internship is an AI-powered Retrieval-Augmented Generation (RAG) system designed to improve enterprise document management and knowledge retrieval (**Lewis et al., 2020**). The application enables users to upload organizational documents, process them automatically, and interact with their contents through a conversational interface. Instead of manually searching through lengthy reports or technical documentation, users can ask questions in natural language and receive responses generated from the relevant information contained within the uploaded documents.

The primary objective of the system is to improve information accessibility while ensuring that responses are generated from trusted organizational documents rather than relying solely on the knowledge of a language model. By combining semantic document retrieval with AI-assisted response generation, the

system provides context-aware, document-grounded answers while supplying only the most relevant document content for response generation.

Document Upload and Processing

Users can upload and manage enterprise documents through a centralized repository. After upload, the system automatically extracts document text, performs preprocessing, divides the content into semantic chunks, and prepares it for retrieval. This automated processing pipeline minimizes manual effort and ensures that documents become searchable once processing has been completed.

Semantic Retrieval and Question Answering

The system retrieves relevant document sections using semantic similarity rather than exact keyword matching, allowing users to ask questions in natural language. The retrieved document content is then provided to a language model, which generates context-aware responses grounded in the uploaded documents. The application also presents document references so that users can verify the origin of the retrieved information.

Conversation Management

The application provides a conversation-oriented interface where users can create multiple conversations, select one or more processed documents as knowledge sources, review previous discussions, and continue asking follow-up questions while preserving conversational context.

System Architecture

The system follows a modular architecture that separates document management, document processing, semantic retrieval, conversational interaction, and background processing into independent components. This design improves maintainability, scalability, and extensibility, allowing future enhancements to be incorporated with minimal impact on the existing system.

Overall, the developed RAG system combines automated document processing, semantic search, and AI-assisted question answering within a unified platform for enterprise knowledge retrieval. The solution improves information accessibility, reduces the effort required to locate relevant knowledge, and provides employees with an efficient and reliable way to interact with organizational documents.

4. Tools & Frameworks

The development of the AI-powered Retrieval-Augmented Generation (RAG) system required a combination of backend technologies, artificial intelligence services, frontend frameworks, and development tools. These technologies were selected to support secure document management, automated document processing, semantic retrieval, conversational interaction, and maintainable software development. Table 1 summarizes the primary technologies used throughout the project.

Category	Technologies	Purpose
Backend Development	Laravel	Web application framework implementing the core business logic
Database	PostgreSQL, pgvector	Storage of application data and semantic vector search

Artificial Intelligence	Embedding Model, Language Model	Semantic retrieval and context-aware response generation
Document Processing	Document Text Extraction, OCR	Extraction of searchable text from digital and scanned documents
Frontend Development	Blade Template Engine, Tailwind CSS, JavaScript	Responsive user interface and interactive user experience
Development & Deployment	Docker, Git, GitHub, Visual Studio Code, Composer, Node.js & npm	Development, dependency management, version control, collaboration, and deployment

Table 1: Technologies Used in the Development of RAG System

4.1. Backend Development

The backend of the system was developed using Laravel, which served as the primary web application framework (**Laravel, 2025**). Its MVC architecture and built-in features for routing, authentication, database interaction, and background job processing provided a structured foundation for implementing document management, conversation handling, and the overall application workflow.

PostgreSQL was selected as the primary relational database because of its reliability and enterprise-level capabilities (**PostgreSQL Global Development Group, 2025**). The integration of the pgvector extension enabled efficient storage and similarity search of document embeddings, allowing the system to retrieve relevant document content based on semantic similarity rather than exact keyword matching. Together, Laravel and PostgreSQL provide a scalable and maintainable backend architecture for the application.

4.2. Artificial Intelligence and Document Processing

The intelligent document retrieval workflow combines document processing technologies with artificial intelligence models. During document processing, uploaded files are converted into searchable text using document text extraction techniques, while Optical Character Recognition (OCR) enables scanned documents to be processed in the same workflow (**Poppler Project, 2025; Tesseract OCR, 2025**).

The extracted content is transformed into vector embeddings using an embedding model and stored within the vector-enabled database. During question answering, a language model generates responses based on the document fragments retrieved through semantic similarity search. This combination forms the core Retrieval-Augmented Generation (RAG) workflow that enables context-aware, document-grounded responses.

4.3. Frontend Development

The user interface was developed using Laravel Blade templates, Tailwind CSS, and JavaScript. Blade supports reusable layouts and modular interface development, while Tailwind CSS provides a consistent and responsive design across the application. JavaScript enhances the user experience by enabling interactive features such as asynchronous communication, dynamic interface updates, and real-time processing status indicators without requiring full page reloads (**Tailwind Labs, 2025; Mozilla Developer Network, 2025**).

4.4. Development and Deployment Tools

Several supporting tools were used throughout the development process to improve productivity and maintain a consistent development environment. Docker was used to containerize the application and its supporting services (**Docker Inc., 2025**), ensuring consistent deployment across different environments. Git and GitHub were used for version control and collaborative development, while Visual Studio Code served as the primary development environment. Composer managed PHP dependencies, and Node.js together with npm handled frontend package management and asset compilation.

5. Specification

5.1. Purpose, Scope, Users, and Constraints

Purpose

The purpose of the AI-powered Retrieval-Augmented Generation (RAG) system is to provide an intelligent document management and knowledge retrieval platform that enables users to obtain accurate information from enterprise documents through natural language interactions. By combining automated document processing, semantic retrieval, and AI-assisted response generation, the system improves knowledge accessibility and reduces the effort required to locate information within large collections of organizational documents.

Scope

The system supports the complete lifecycle of enterprise document management and intelligent knowledge retrieval. Users can upload supported document formats, manage their documents, monitor processing status, and interact with one or more processed documents through a conversational interface.

The platform provides automated document processing, semantic retrieval, AI-assisted question answering, conversation management, and document status tracking. The current implementation focuses on enterprise document retrieval and excludes features such as enterprise authentication integration, advanced administrative analytics, and large-scale distributed deployment.

Intended Users

The primary users of the system are employees and knowledge workers who require efficient access to technical documents, manuals, reports, and other organizational knowledge sources. The system also supports administrative users responsible for user management, monitoring document processing activities, and maintaining overall system operation.

Constraints

The effectiveness of the system depends on the quality of uploaded documents, and low-quality or poorly scanned files may reduce retrieval accuracy. The platform supports only predefined document formats and relies on background processing before uploaded documents become available for querying. The system is intended for enterprise environments where secure access, document confidentiality, and efficient resource utilization are required.

5.2. Functional Requirements

The functional requirements define the core capabilities of the AI-powered Retrieval-Augmented Generation (RAG) system (**ISO/IEC/IEEE 29148, 2011**).

User Authentication and Access Management

The system should provide secure user authentication and role-based authorization. Authenticated users shall have access only to their own documents and conversations, while administrators shall be granted additional management privileges.

Document Management

The system shall allow users to upload, view, organize, and delete supported documents. Uploaded documents shall be securely associated with the authenticated user, and relevant metadata, including processing status, shall be maintained.

Automated Document Processing

The system shall automatically process uploaded documents by extracting text, preprocessing content, generating semantic chunks, and preparing them for retrieval through asynchronous background processing.

Processing Status Monitoring

The system should provide real-time visibility of document processing stages and indicate when documents are ready for querying or if processing has failed.

Intelligent Document Retrieval

The system should retrieve relevant document content using semantic similarity rather than exact keyword matching to improve retrieval accuracy.

AI-Assisted Question Answering

The system shall generate context-aware responses based on retrieved document content while minimizing unsupported or irrelevant answers.

Multi-Document Query Support

The system should allow users to select one or more processed documents as knowledge sources for a conversation and retrieve information across the selected documents.

Conversation Management

The system shall allow users to create, continue, rename, and review conversations while preserving conversation history.

Search Result References

The system shall associate generated responses with the corresponding document references to improve transparency and enable users to verify retrieved information.

Administrative Functions

The system should provide administrative capabilities for user management, document processing supervision, background job monitoring, and overall system maintenance.

5.3. Non-functional Requirements

The AI-powered Retrieval-Augmented Generation (RAG) system shall satisfy the following non-functional requirements to ensure reliable and efficient operation within an enterprise environment.

Performance

The system shall provide responsive document management and conversational interactions by executing document processing asynchronously and generating query responses within an acceptable response time.

Scalability

The architecture shall support future growth in users, documents, and conversations without requiring significant architectural changes.

Security

The system shall protect user documents and conversations through authentication, authorization, and role-based access control, ensuring complete isolation of user data.

Reliability

The platform shall provide reliable document processing and retrieval through background processing, error handling, and status reporting mechanisms.

Availability

The system shall remain available during normal operation while allowing users to continue working during long-running background processing tasks.

Usability

The user interface shall provide an intuitive and consistent experience that enables users to upload documents, monitor processing, and interact with the conversational interface with minimal effort.

Maintainability

The application shall follow a modular architecture that simplifies maintenance, feature enhancement, and long-term software evolution.

Compatibility

The system shall support commonly used enterprise document formats and operate through modern web browsers without requiring additional client-side software.

Extensibility

The architecture shall support the integration of additional document formats, retrieval techniques, administrative capabilities, and AI features with minimal modification to the existing system.

5.4. User Stories

User Story 1: User Authentication

As a user,

I want to securely log in to the system,
So that I can access my personal workspace and interact with my documents.

Acceptance Criteria:

- Only registered users can successfully authenticate.
- Users can access only their own documents and conversations.
- Invalid login attempts display an appropriate error message.

User Story 2: Uploading Documents

As a user,
I want to upload documents to the system,
So that they can be processed and used as knowledge sources for future conversations.

Acceptance Criteria:

- The system accepts supported document formats and stores them securely.
- Document processing starts automatically after a successful upload.
- The user receives confirmation that the upload was successful.

User Story 3: Monitoring Document Processing

As a user,
I want to monitor the processing status of my uploaded documents,
So that I know when they are ready to be queried.

Acceptance Criteria:

- The system displays the current processing status of each document.
- Processing status is updated automatically until completion.
- Processing failures are reported appropriately.

User Story 4: Managing Documents

As a user,
I want to view and manage my uploaded documents,
So that I can organize my knowledge base efficiently.

Acceptance Criteria:

- Users can view document details and processing status.
- Users can delete documents that are no longer required.
- Changes are reflected immediately after the operation.

User Story 5: Starting a Conversation

As a user,
I want to start a new conversation by selecting one or more processed documents,
So that I can retrieve information from the relevant knowledge sources.

Acceptance Criteria:

- Only processed documents can be selected.

- Users can select one or multiple documents for a conversation.
- The selected documents become the conversation's knowledge source.

User Story 6: Asking Questions

As a user,

I want to ask questions in natural language,

So that I can quickly obtain relevant information without manually searching through documents.

Acceptance Criteria:

- The system accepts natural language queries.
- Responses are generated from the selected documents.
- If no relevant information is found, the user is informed.

User Story 7: Retrieving Information from Multiple Documents

As a user,

I want to retrieve information from multiple documents within a single conversation,

So that I can obtain comprehensive answers from different knowledge sources.

Acceptance Criteria:

- Users can select multiple processed documents.
- Retrieval considers only the selected documents.
- Responses are generated using the combined retrieved information.

User Story 8: Viewing Response References

As a user,

I want to view the document references associated with generated responses,

So that I can verify the source of the retrieved information.

Acceptance Criteria:

- Responses include references to the relevant document sources.
- Users can identify the origin of the retrieved information.

User Story 9: Managing Conversation History

As a user,

I want to access my previous conversations,

So that I can continue earlier discussions without losing context.

Acceptance Criteria:

- Previous conversations are stored and accessible.
- Selecting a conversation restores its message history.
- Users can continue existing conversations.

User Story 10: Administering the System

As an administrator,

I want to manage users and monitor system activities,
So that the platform remains secure, reliable, and properly maintained.

Acceptance Criteria:

- Administrative functions are accessible only to authorized users.
- Administrators can manage users and monitor system activities.
- System resources and document processing can be supervised.

5.5. Use Case Diagrams and Use Case Descriptions

The use case diagram illustrates the primary interactions between users, administrators, and the AI-powered document retrieval system. It summarizes the core functionalities available to each actor, including document management, document processing, conversational knowledge retrieval, and system administration.

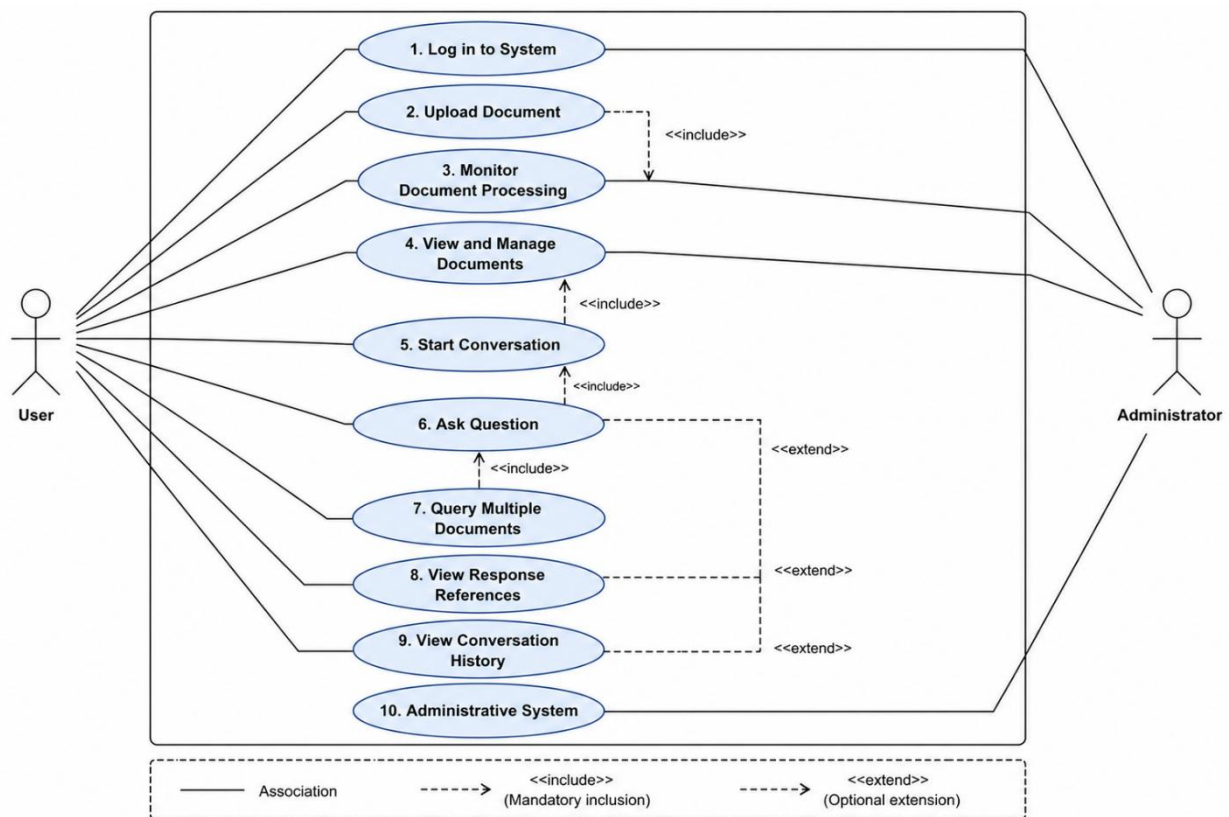


Figure 1: Overall Use Case Diagram of the AI-powered Document Retrieval System

The diagram shows that users can authenticate, upload and manage documents, monitor document processing, start conversations, ask questions, retrieve information from multiple documents, view response references, and access previous conversations. Administrators have additional privileges for user management and system monitoring.

"Query Multiple Documents" extends "Ask Question" because it represents a specialized form of question answering involving multiple knowledge sources. Similarly, "View Response References" extends "Ask Question" because references are displayed after a response is generated. "View

Conversation History" remains an independent use case since users may review previous conversations without asking a new question.

Use Case 1: Log in to System

Actors: User, Administrator

Description: Authenticates a user before granting access to the system.

Preconditions:

- The user must have a registered account.
- The system must be available.

Postconditions:

- The user is authenticated and redirected to the appropriate dashboard.

Normal Flow:

1. The user opens the login page.
2. The user enters login credentials.
3. The system validates the credentials.
4. The user is granted access and redirected to the main dashboard.

Alternative Flows and Exceptions:

- Invalid credentials display an error message.
- Inactive accounts are denied access.
- If the system is unavailable, login cannot proceed.

Use Case 2: Upload Document

Actors: User

Description: Uploads a supported document for processing.

Preconditions:

- The user must be logged in.
- The uploaded file must be in a supported document format.

Postconditions:

- The document is uploaded successfully.
- The document processing workflow is started.

Normal Flow:

1. The user selects and uploads a document.
2. The system validates the file.
3. The document is stored.
4. Background processing is initiated.
5. A confirmation is displayed.

Alternative Flows and Exceptions:

- Unsupported files are rejected.

- Invalid file size displays an error.
- Upload failures leave the document unprocessed.

Use Case 3: Monitor Document Processing

Actors: User, Administrator

Description: Displays the processing progress of uploaded documents.

Preconditions:

- The user must be logged in.
- At least one document must have been uploaded.

Postconditions:

- The current processing status is displayed.

Normal Flow:

1. The user opens the document list.
2. The system displays processing status.
3. Status updates until processing is completed.

Alternative Flows and Exceptions:

- Processing failures are reported.
- Documents still being processed cannot be queried.
- If status retrieval fails, the last known status is shown.

Use Case 4: View and Manage Documents

Actors: User, Administrator

Description: Allows users to view and manage uploaded documents.

Preconditions:

- The user must be logged in.
- The user must have uploaded documents.

Postconditions:

- Document information is updated after any management action.

Normal Flow:

1. The user opens the document management page.
2. The system displays uploaded documents.
3. The user views or deletes a document.
4. The system updates the document list.

Alternative Flows and Exceptions:

- An empty state is shown when no documents exist.
- Deletion requires confirmation.
- Errors are displayed if deletion fails.

Use Case 5: Start Conversation

Actors: User

Description: Creates a new conversation using selected processed documents.

Preconditions:

- The user must be logged in.
- At least one document must be processed and ready for querying.

Postconditions:

- A conversation is created with the selected documents.

Normal Flow:

1. The user selects one or more documents.
2. A new conversation is created.
3. The chat interface opens.

Alternative Flows and Exceptions:

- Conversation creation is blocked if no processed documents exist.
- Selecting unprocessed documents generates a warning.
- Errors are displayed if creation fails.

Use Case 6: Ask Question

Actors: User

Description: Retrieves an answer using the selected document(s).

Preconditions:

- The user must be logged in.
- A conversation must exist.

Postconditions:

- The question and generated response are stored.

Normal Flow:

1. The user submits a question.
2. The system retrieves relevant document content.
3. A response is generated.
4. The response is displayed.

Alternative Flows and Exceptions:

- If no relevant content is found, the user is informed.
- Response generation failures display an error.
- Queries cannot continue if selected documents are unavailable.

Use Case 7: Query Multiple Documents

Actors: User

Description: Retrieves information across multiple selected documents.

Preconditions:

- The user must be logged in.
- Multiple processed documents are selected.

Postconditions:

- A combined response is generated.

Normal Flow:

1. The user selects multiple documents.
2. A question is submitted.
3. The system retrieves information from all selected documents.
4. A combined response is generated.

Alternative Flows and Exceptions:

- Unavailable documents are ignored.
- If no relevant information exists, an appropriate message is displayed.
- Retrieval may be limited when too many documents are selected.

Use Case 8: View Response References

Actors: User

Description: Displays document references associated with generated responses.

Preconditions:

- The user must be logged in.
- A response must have been generated.

Postconditions:

- Related document references are displayed.

Normal Flow:

1. The system generates a response.
2. Related references are displayed.
3. The user reviews the sources.

Alternative Flows and Exceptions:

- Responses may be displayed without references.
- Deleted source documents may no longer be accessible.
- Retrieval failures prevent references from being shown.

Use Case 9: View Conversation History

Actors: User

Description: Displays previous conversations and allows them to be continued.

Preconditions:

- The user must be logged in.
- At least one conversation must exist.

Postconditions:

- The selected conversation is restored.

Normal Flow:

1. The user opens the conversation history.
2. The system displays previous conversations.
3. The selected conversation is loaded.

Alternative Flows and Exceptions:

- An empty state is displayed when no conversations exist.
- Loading failures display an error.
- Missing related documents generate a warning.

Use Case 10: Administrative System

Actors: Administrator

Description: Enables administrators to monitor and manage the platform.

Preconditions:

- The administrator must be logged in.
- The administrator must have appropriate privileges.

Postconditions:

- Requested administrative actions are completed.

Normal Flow:

1. The administrator opens the dashboard.
2. The system displays operational information.
3. Administrative actions are performed.
4. The system applies the changes.

Alternative Flows and Exceptions:

- Unauthorized users are denied access.
- Errors are displayed if data cannot be loaded.
- Failed administrative actions are logged.

5.6. External Interfaces

The AI-powered document retrieval system interacts with several external components that support document processing, semantic retrieval, response generation, data storage, and application deployment. These interfaces enable the system to communicate with external services while keeping the core application independent of their internal implementations. The following subsections describe the primary external interfaces used throughout the system.

Database Interface

The system uses a PostgreSQL database with vector storage capabilities as its primary persistence layer. The database stores user information, uploaded document metadata, processed document chunks, conversation history, and vector embeddings generated during document processing. By separating structured application data from the application logic, the database provides efficient storage, retrieval, and indexing of information required by the retrieval pipeline.

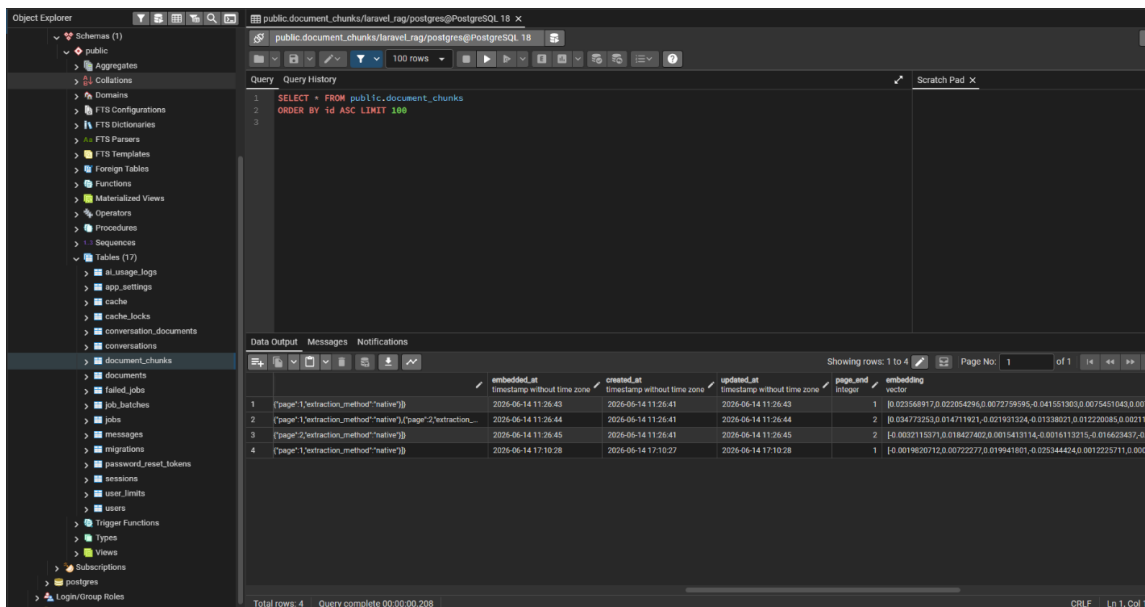


Figure 2: PostgreSQL Database Interface

Embedding Model Interface

The embedding model interface is responsible for transforming processed document chunks into high-dimensional vector representations. These embeddings capture the semantic meaning of the document content and enable similarity-based retrieval during user queries. The application communicates with the embedding service during the document processing pipeline, and the generated vectors are subsequently stored in the database for future retrieval operations.

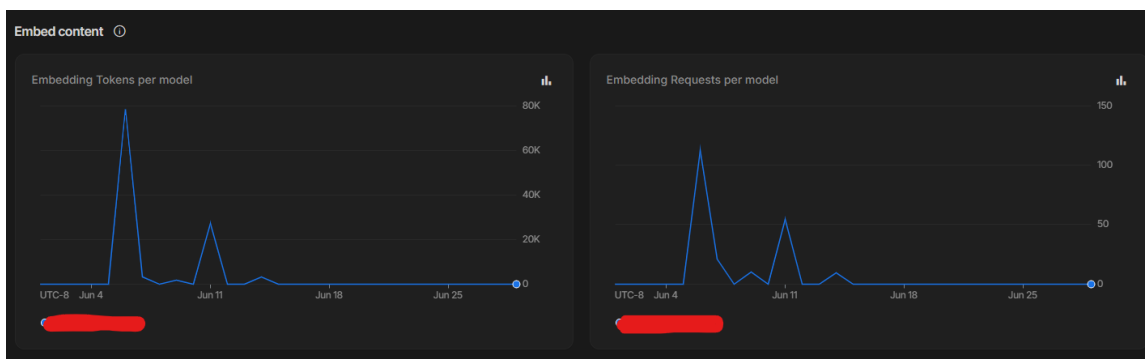


Figure 3: Embedding Model Interface Used for Document Vector Generation

Language Model Interface

The language model interface is used during the question-answering process. After relevant document content has been retrieved, the selected context together with the user's question is forwarded to the language model, which generates a context-aware response. The application exchanges prompts and responses through this interface while ensuring that the generated answer is based on the retrieved document context.

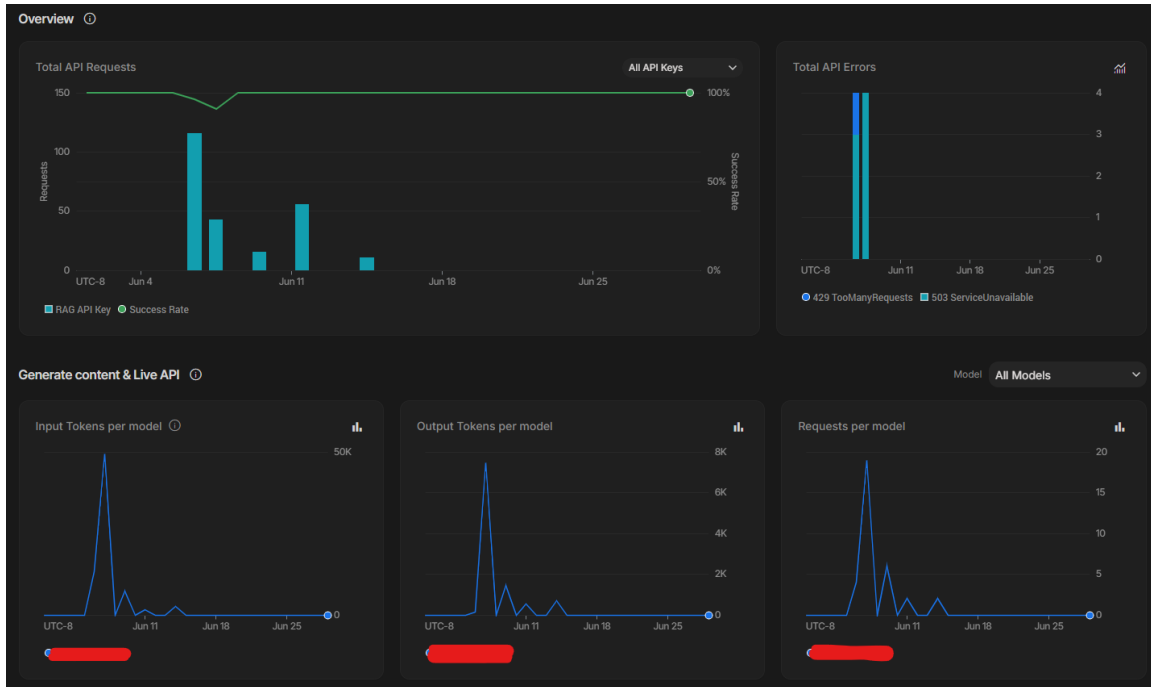


Figure 4: Language Model Interface Used for Context-Aware Response Generation

Background Processing Interface

Document processing is performed asynchronously through a background job processing mechanism. After a document is uploaded, the system automatically schedules a series of processing jobs responsible for text extraction, document chunking, embedding generation, and database storage. Executing these operations in the background prevents long-running tasks from blocking user interactions and allows users to monitor processing progress while the application continues operating normally.

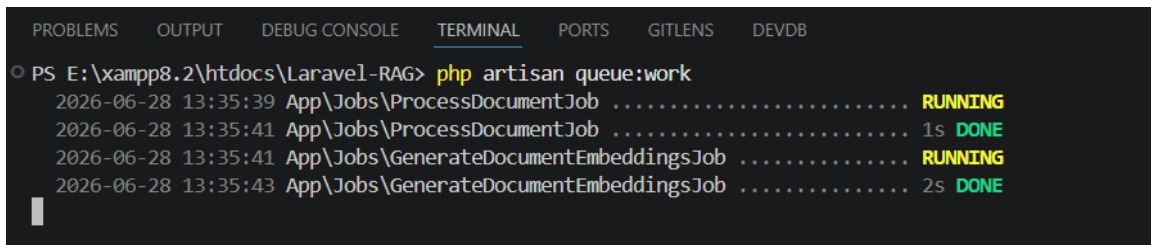


Figure 5: Background Job Processing Interface

Containerization Interface

To simplify development, deployment, and environment consistency, the application is containerized using Docker. Individual containers are used to isolate the web application, database, web server, and background processing services while allowing them to communicate through a shared container network. This containerized architecture ensures that the application can be deployed consistently across different development and production environments with minimal configuration changes.

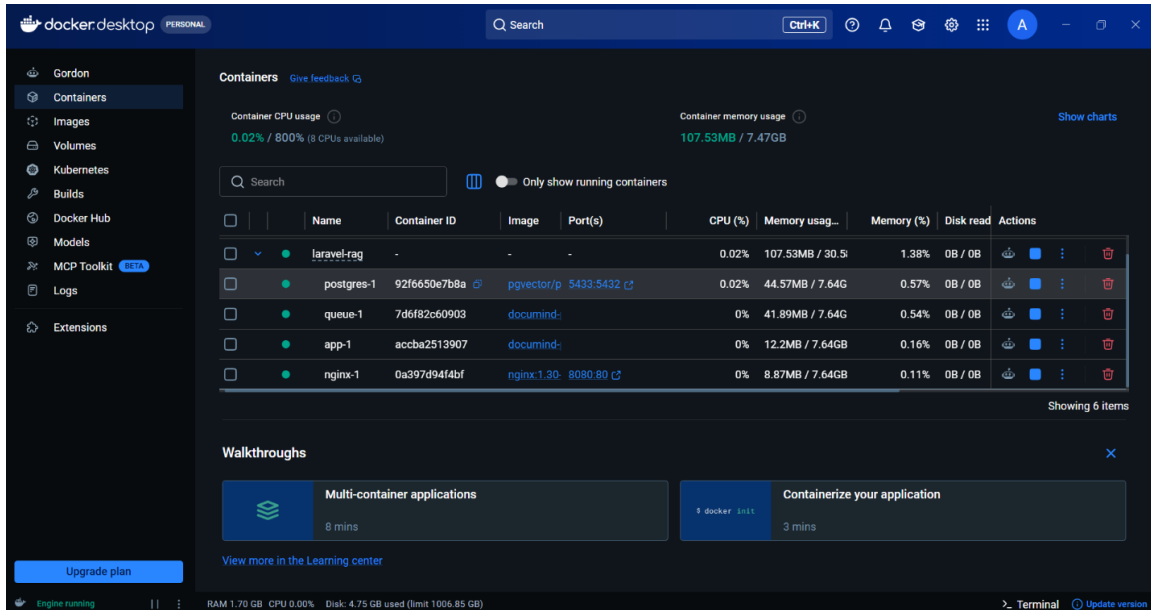


Figure 6: Docker Container Environment

5.7. User Interface Mockups

The user interface of the AI-powered document retrieval system was designed to provide a simple and intuitive experience for both end users and administrators. The application follows a dashboard-based layout that enables users to manage documents, monitor processing progress, and interact with the conversational retrieval system efficiently. Each interface was developed with a focus on usability, consistency, and ease of navigation, ensuring that users can perform their tasks with minimal complexity. This section presents the primary interfaces implemented during the development of the system.

5.7.1. Login Interface

The login interface provides secure authentication before users can access the application. Only registered users with valid credentials are permitted to enter the workspace. Upon successful authentication, users are redirected to their respective dashboards according to their assigned roles. This interface serves as the primary security mechanism for protecting uploaded documents and conversation data.

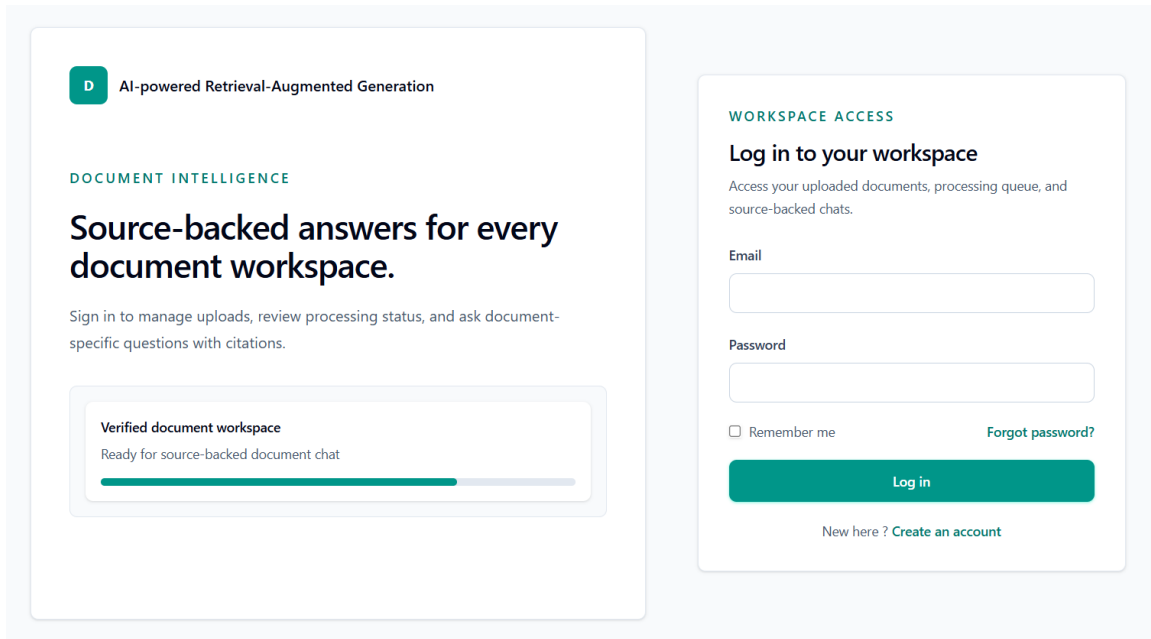


Figure 7: Login Interface

5.7.2. User Dashboard

The user dashboard serves as the central workspace after successful authentication. It provides an overview of the user's document repository by displaying key statistics such as the total number of uploaded documents, documents ready for querying, documents currently being processed, and the total number of questions asked. The dashboard also presents recently uploaded documents, enabling users to quickly monitor document availability and access frequently used resources.

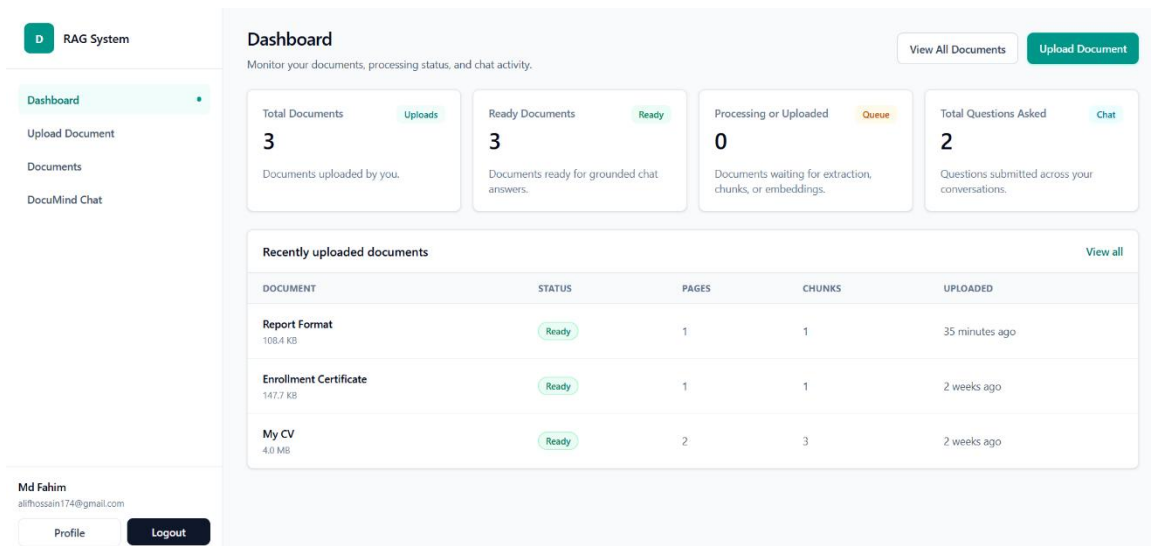


Figure 8: User Dashboard

5.7.3. Document Upload Interface

The document upload interface enables users to submit supported document formats for processing. Users may optionally provide document metadata, such as a title and description, before uploading the

file. After submission, the application automatically initiates the background processing workflow, including text extraction, document chunking, and embedding generation. The interface also displays recently uploaded documents together with their current processing status.

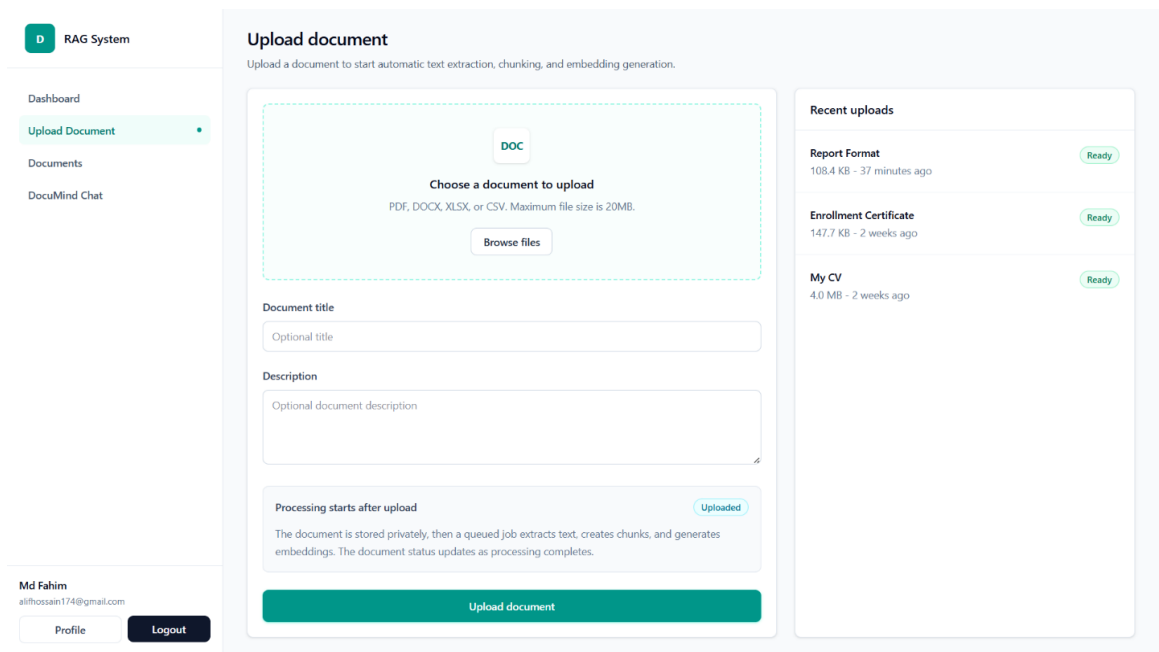


Figure 9: Document Upload Interface

5.7.4. Document Details and Processing Status

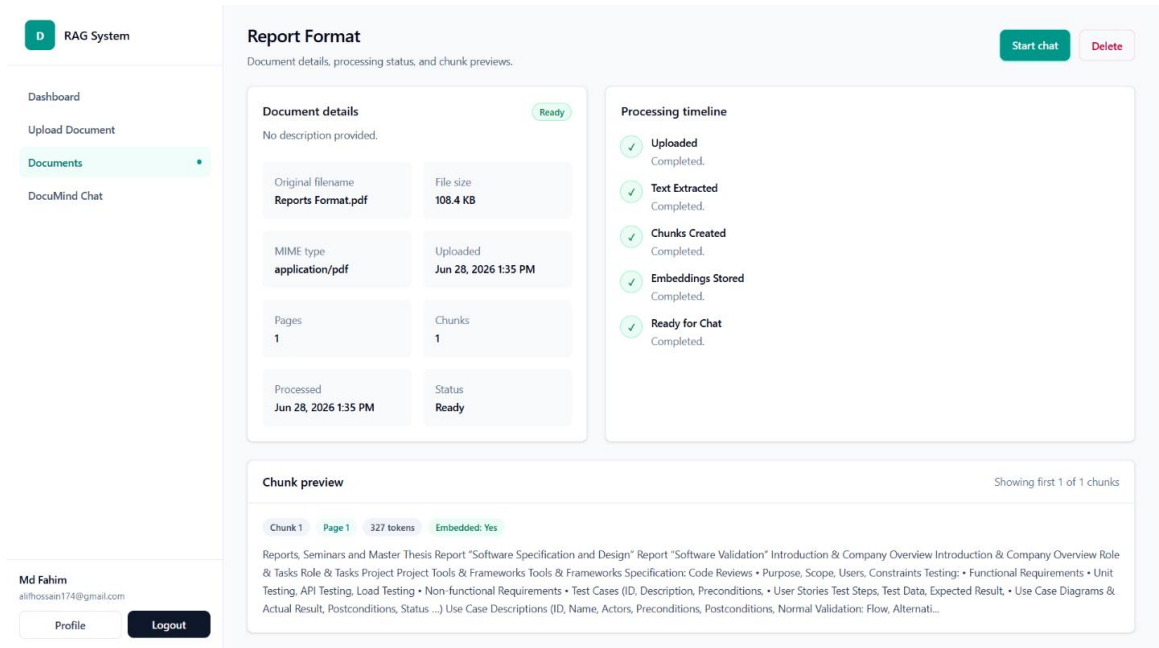


Figure 10: Document Details and Processing Status Interface

The document details interface provides comprehensive information about an uploaded document together with its processing timeline. Users can monitor each stage of the document processing pipeline,

including upload completion, text extraction, chunk creation, embedding generation, and readiness for conversational querying. The interface additionally displays document metadata and previews of the generated document chunks, allowing users to verify the extracted content before initiating conversations with the system.

5.7.5. Conversation Interface

The conversation interface represents the primary functionality of the AI-powered document retrieval system. Users can create new conversations by selecting one or multiple processed documents and subsequently ask natural language questions based on the selected knowledge sources. The interface displays AI-generated responses together with document citations, suggested follow-up questions, and conversation controls, providing an intuitive and interactive environment for document-based knowledge retrieval.

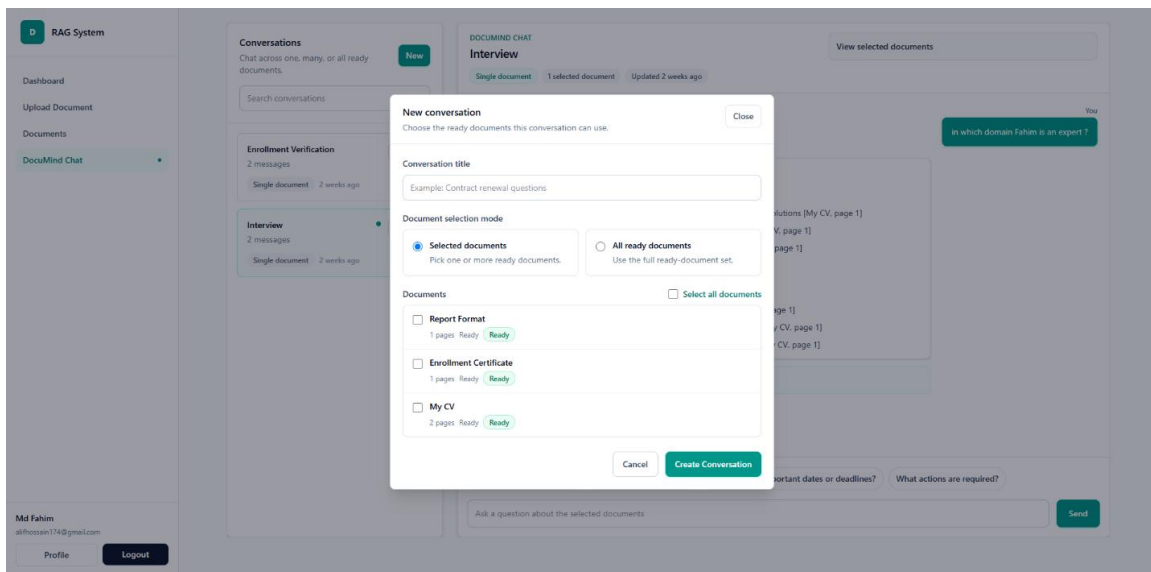


Figure 11: Conversation Interface

5.7.6. Conversation History

The conversation history interface enables users to revisit previous discussions and continue existing conversations without losing context. Conversations are presented in a searchable list together with relevant metadata, including the document scope and the last interaction time. This functionality supports long-term knowledge management by allowing users to review previously generated responses and continue earlier conversations whenever required.

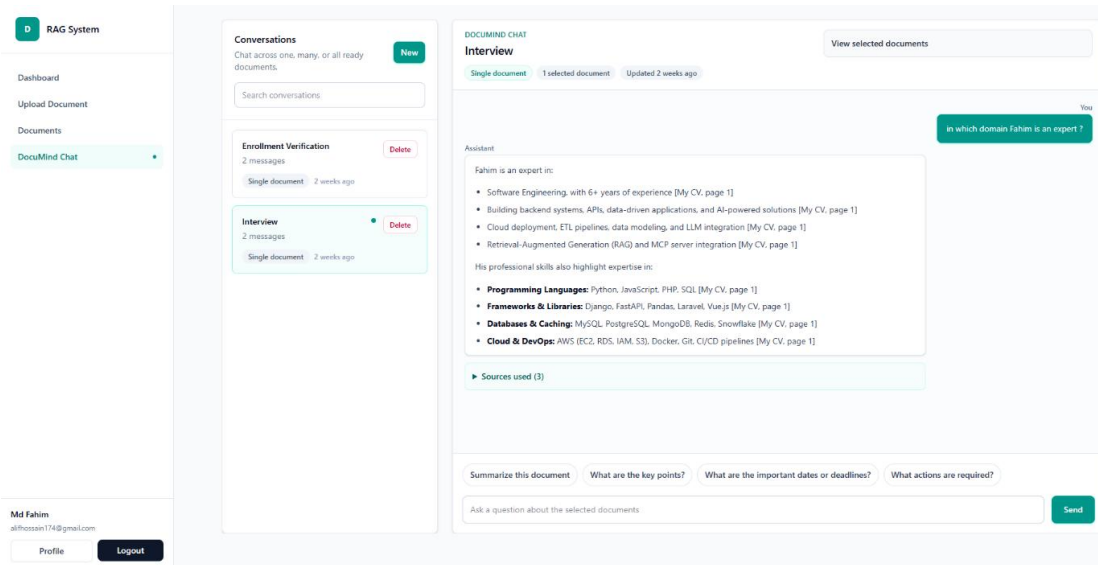


Figure 12: Conversation History Interface

5.7.7. Administrative Dashboard

The administrative dashboard provides centralized management and monitoring capabilities for the overall system. It presents operational statistics related to users, uploaded documents, document processing, and generated document chunks through a single interface. Administrators can also access dedicated modules for user management, document supervision, usage limits, background queue monitoring, failed job management, and system configuration. By consolidating these administrative functions into a unified dashboard, the interface supports efficient system supervision and helps maintain the reliable operation of the AI-powered document retrieval system.

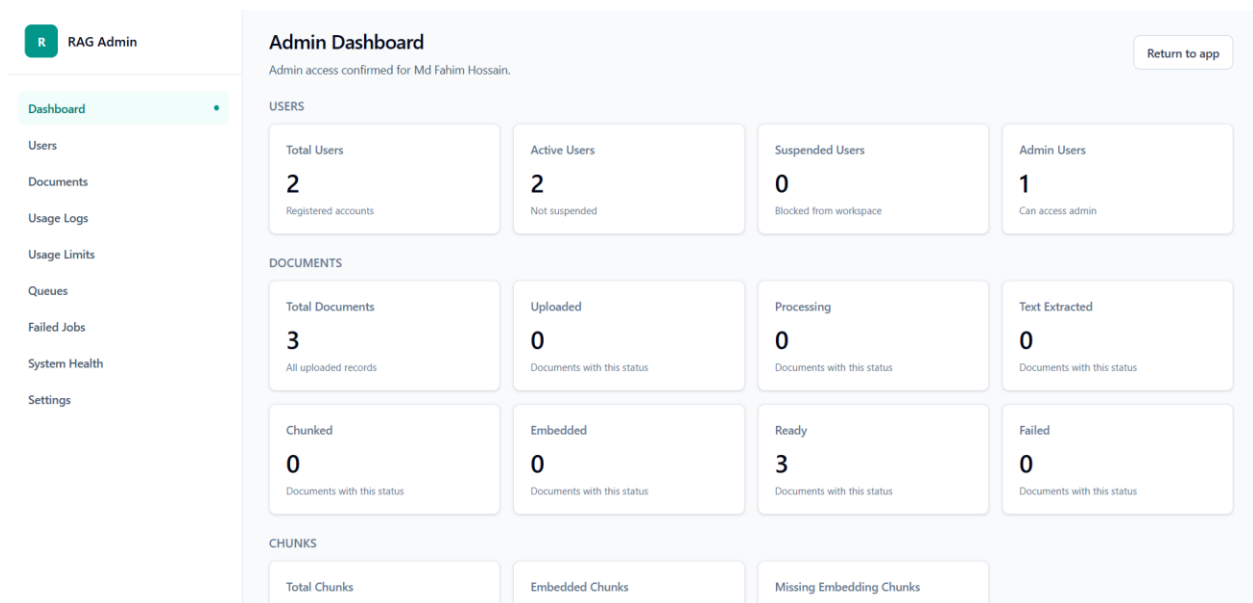


Figure 13: Administrative Dashboard

6. Design

6.1. System Architecture

The architecture of the AI-powered document retrieval system was designed with the objectives of modularity, scalability, and maintainability. Since the application combines traditional web application features with artificial intelligence capabilities, the architecture separates the responsibilities of user interaction, business logic, document processing, knowledge retrieval, and data storage into independent layers. This separation allows each component to evolve independently while maintaining a clear and well-defined communication flow throughout the system.

The system follows a layered architecture in which users interact with the application through a web-based interface developed using Laravel Blade templates and Tailwind CSS. User requests are processed by the Laravel application, which acts as the central controller responsible for authentication, document management, conversation management, and coordination of the document processing pipeline. This approach keeps the presentation layer independent from the underlying processing logic, resulting in a clean and maintainable system structure.

A dedicated document processing pipeline is responsible for transforming uploaded documents into searchable knowledge. When a user uploads a supported document, the system stores the original file and initiates a series of background processing tasks. These tasks include text extraction, optional optical character recognition for scanned documents, text cleaning, document chunking, embedding generation, and storage of vector representations within the database. Executing these operations asynchronously prevents long waiting times for users while ensuring that computationally intensive tasks do not affect the responsiveness of the application.

The knowledge retrieval process is separated from document processing and is only activated when a user submits a question. Instead of searching entire documents directly, the system generates an embedding for the user's query and performs a semantic similarity search against the stored document embeddings. Only the most relevant document chunks are retrieved and combined with the user's question before being forwarded to the language model. This Retrieval-Augmented Generation (RAG) approach improves answer relevance while reducing unnecessary processing (Lewis et al., 2020) and limiting the amount of document content sent during inference.

The persistence layer consists of a PostgreSQL database enhanced with vector storage capabilities. Besides managing traditional relational data such as users, conversations, uploaded documents, and processing status, the database also stores vector embeddings for semantic search. This unified storage approach simplifies data management while providing efficient retrieval of relevant document content during question answering.

Several external services are integrated into the architecture to provide specialized capabilities. Text extraction libraries process uploaded documents, OCR tools handle scanned files when necessary, embedding models convert text into numerical vector representations, and a language model generates context-aware responses. These services are accessed through dedicated service classes, ensuring loose coupling between the application core and external dependencies. Such abstraction improves

maintainability and allows individual services to be replaced in the future with minimal impact on the overall system.

Overall, the architecture provides a clear separation between user interaction, document processing, semantic retrieval, artificial intelligence services, and data persistence. This modular design improves maintainability, supports future extensions, and enables the system to efficiently process enterprise documents while delivering context-aware responses through the Retrieval-Augmented Generation workflow.

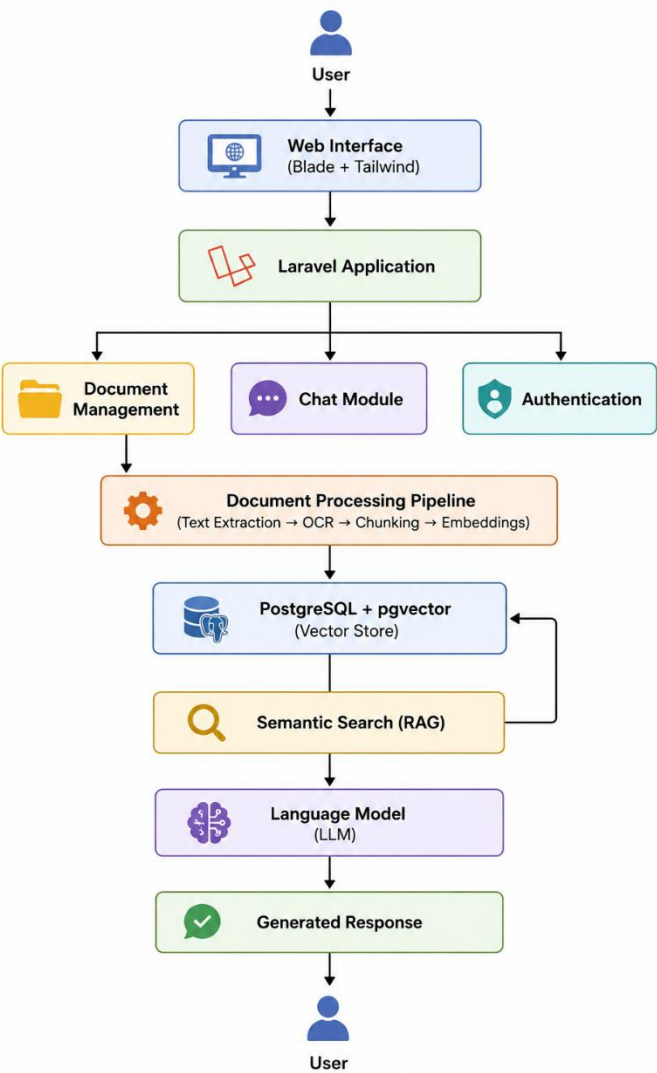


Figure 14: High-Level Architecture of the AI-Powered Enterprise RAG System

6.2. Component Diagram

The component diagram provides a structural view of the major software modules that make up the AI-powered Retrieval-Augmented Generation (RAG) system. While the system architecture presented in the previous section illustrates the overall workflow of the application, the component diagram focuses on the internal organization of the software and the relationships between its primary functional modules.

The system follows a modular design in which each component is responsible for a specific set of functionalities. This separation of responsibilities reduces coupling between different parts of the application and improves maintainability, scalability, and extensibility. Individual components communicate through well-defined interfaces while remaining independent of each other's internal implementations.

At the center of the application is the Laravel Application Core, which coordinates communication between all other components. It manages incoming user requests, executes business logic, invokes document processing services, performs semantic retrieval, and communicates with external artificial intelligence services.

The User Interface component provides the web-based frontend through which users upload documents, manage conversations, and interact with the AI assistant. User requests are forwarded to the application core, while generated responses and document information are returned to the interface for presentation.

The Authentication component manages user authentication and authorization. It verifies user identities, enforces access control, and ensures that users can access only their own documents and conversations, while administrators are granted additional management capabilities.

The Document Management component is responsible for handling document uploads, storage, metadata management, and document lifecycle operations. Once a document is uploaded successfully, this component initiates the document processing workflow.

The Document Processing component performs all operations required to prepare uploaded documents for semantic retrieval. These operations include text extraction, optical character recognition when necessary, text preprocessing, document chunking, and embedding generation. Processing tasks are executed asynchronously through the background job system to improve application responsiveness.

The Semantic Retrieval component performs similarity search against the stored document embeddings. After converting the user's question into an embedding, it retrieves the most relevant document chunks from the vector database and prepares the contextual information required for response generation.

The Conversation Management component maintains conversation history, stores user messages and AI responses, manages document selections for each conversation, and preserves conversational context across multiple interactions.

The AI Integration component provides communication with external artificial intelligence services responsible for embedding generation and context-aware response generation. By isolating these interactions within a dedicated component, the application remains independent of specific AI service implementations, allowing future replacement or extension with minimal architectural changes.

Finally, the Database component serves as the persistent storage layer for the application. It stores user accounts, uploaded documents, document chunks, vector embeddings, conversations, processing status, and other application data required for system operation.

Overall, the component-based architecture enables the application to separate user interaction, document processing, semantic retrieval, conversation management, and persistent storage into independent modules. This modular organization simplifies maintenance, improves scalability, and provides a solid foundation for future system enhancements.

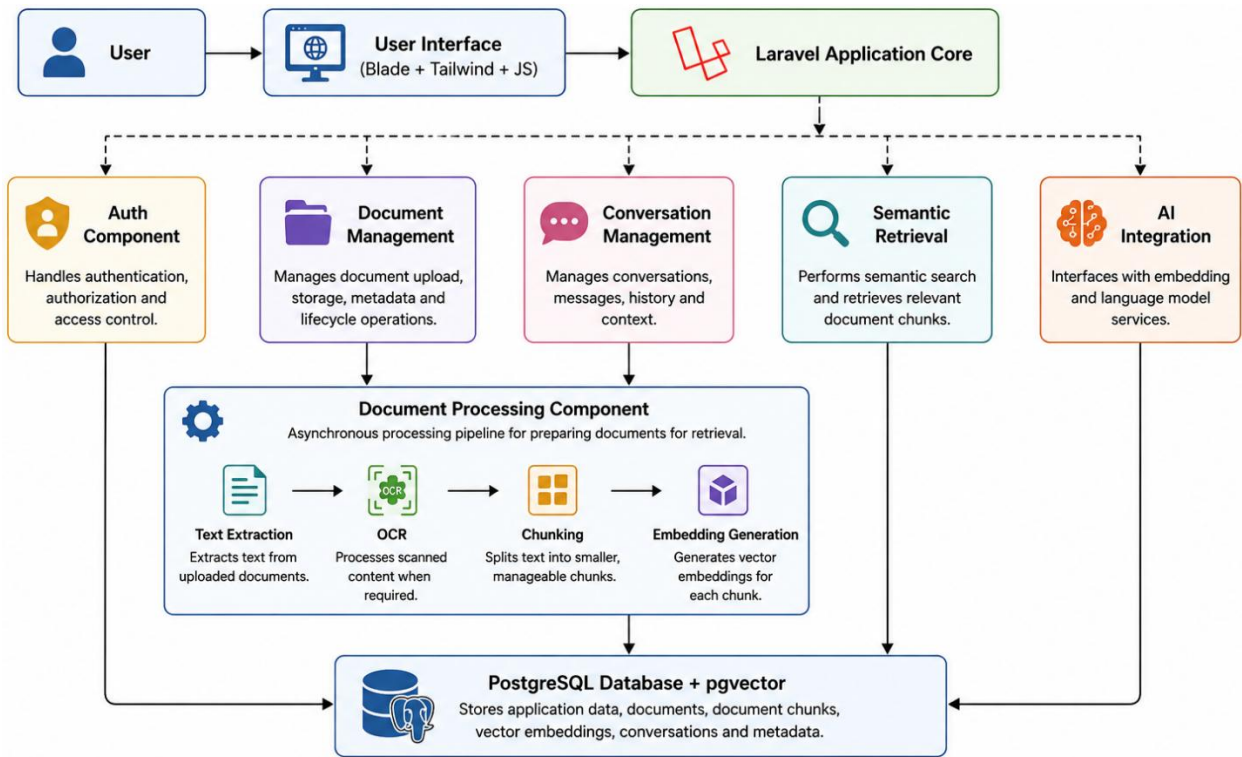


Figure 15: UML Component Diagram of the AI-Powered Enterprise RAG System

6.3. Class Diagram

The class diagram presented in Figure 6.3 illustrates the static structure of the AI-powered Retrieval-Augmented Generation (RAG) system by identifying the primary classes, their key attributes, and the relationships between them. While the component diagram describes the interaction between the major software modules, the class diagram focuses on the object-oriented design of the application and the relationships between its core entities and service classes.

The User class represents authenticated users of the application and serves as the starting point of the system's data model. Each user can upload multiple documents and create multiple conversations with the AI assistant. This one-to-many relationship allows every user to maintain an independent collection of documents and conversation history while ensuring proper separation of user data.

Uploaded documents are represented by the Document class, which stores essential information such as the document name, processing status, and file location. During the document processing pipeline, each document is divided into multiple DocumentChunk objects. Every document chunk contains a portion of the extracted text together with its corresponding embedding vector and metadata required for

semantic search. Since a single document can generate many chunks, the relationship between Document and DocumentChunk is one-to-many.

The Conversation class represents an individual chat session initiated by a user. Each conversation consists of multiple Message objects that store both user queries and AI-generated responses. This separation enables the application to preserve the complete conversation history and maintain contextual information throughout an ongoing discussion.

To support conversations involving multiple uploaded documents, the ConversationDocument class functions as an associative entity between Conversation and Document. This many-to-many relationship allows users to select one or more documents as knowledge sources for a conversation while enabling the same document to participate in multiple independent conversations without duplicating stored information.

Besides the core domain entities, the system contains several service classes that implement the application's business logic. The DocumentProcessor service manages document processing operations, including text extraction, preprocessing, and document chunk generation. The EmbeddingService converts both document chunks and user queries into vector embeddings that enable semantic similarity search. The AIChatService coordinates the retrieval workflow by obtaining the most relevant document chunks and communicating with the language model to generate context-aware responses. Separating these responsibilities into dedicated service classes improves modularity while keeping the business logic independent from the persistent data model.

Overall, the class diagram demonstrates how the system combines object-oriented design principles with a modular service architecture. The clear separation between persistent entities and service classes improves maintainability, simplifies future extensions, and provides a well-structured foundation for document processing, semantic retrieval, and AI-assisted knowledge retrieval within the enterprise RAG system.

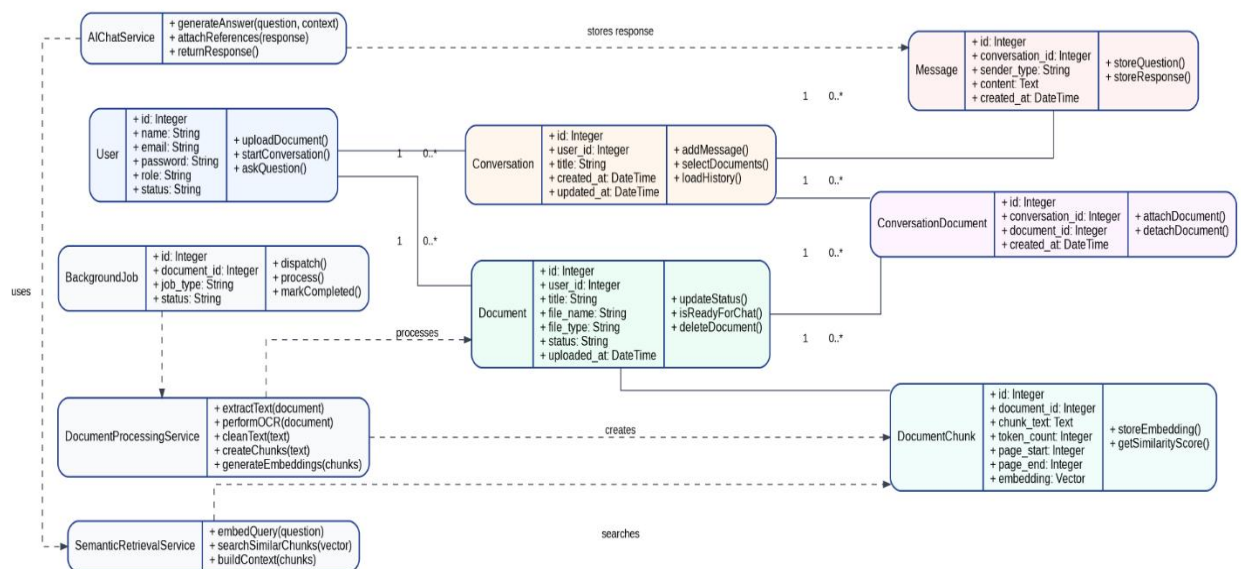


Figure 16: UML Class Diagram of the AI-Powered Enterprise RAG System

6.4. Sequence Diagrams

Sequence diagrams describe the dynamic interactions between the different components of the AI-powered Retrieval-Augmented Generation (RAG) system during the execution of key business processes. Unlike the class diagram, which illustrates the static structure of the application, sequence diagrams emphasize the chronological order in which system components communicate to complete a specific task.

The developed system contains two primary workflows that represent its core functionality. The first workflow describes how uploaded documents are processed and prepared for semantic retrieval, while the second workflow illustrates how user questions are answered through the Retrieval-Augmented Generation process. Together, these sequence diagrams provide a comprehensive view of the interaction between the user interface, application services, document processing pipeline, database, and artificial intelligence services during normal system operation.

Document Upload & Processing

The document upload and processing workflow begins when a user uploads a supported document through the web interface. After receiving the upload request, the Laravel application validates the document and stores the original file together with its associated metadata. Once the upload has been completed successfully, the application initiates the background document processing workflow.

The document processing component performs a sequence of operations to prepare the document for semantic retrieval. First, textual content is extracted from the uploaded document. If the document contains scanned pages rather than searchable text, the OCR component is used to convert the images into machine-readable text. The extracted content is subsequently cleaned and divided into multiple semantic chunks that preserve contextual meaning while remaining suitable for efficient retrieval.

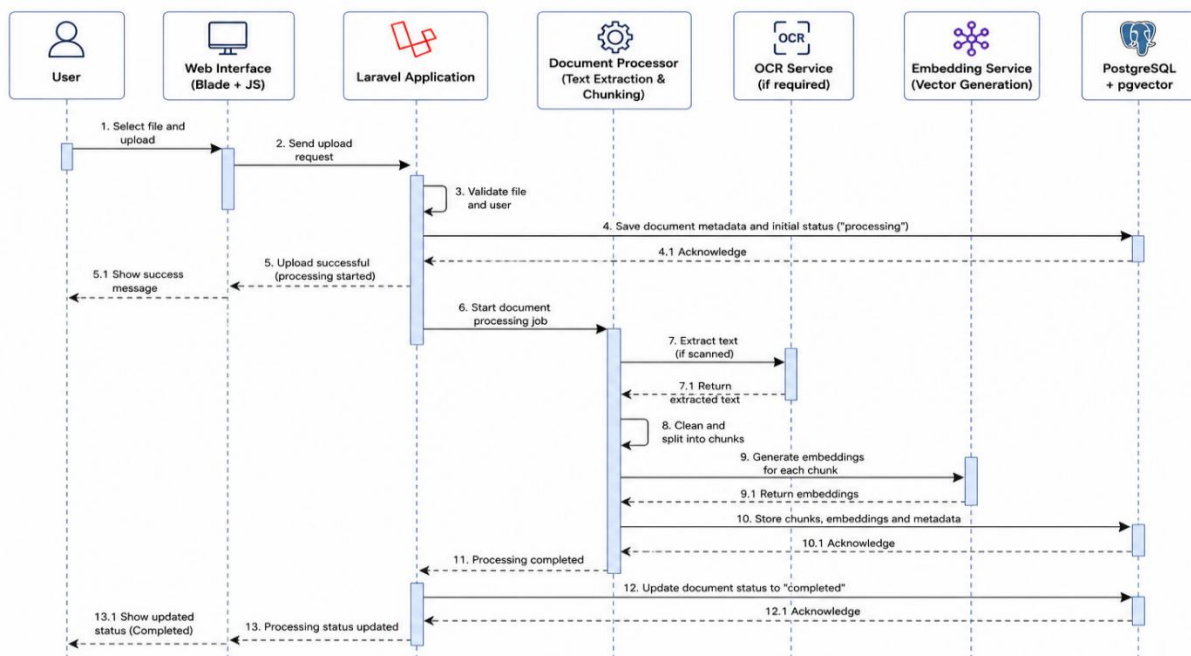


Figure 17: Sequence Diagram for Document Upload and Processing Workflow

Each generated document chunk is then forwarded to the embedding service, which converts the textual content into numerical vector representations. These embeddings, together with the document chunks and related metadata, are stored within the PostgreSQL database enhanced with vector storage capabilities. Throughout this process, the application continuously updates the processing status, allowing users to monitor the progress through the user interface.

Once all processing stages have been completed successfully, the document status is updated to indicate that it is ready for conversational querying. From this point onward, the processed document becomes available as a knowledge source for semantic retrieval during user conversations.

Question Answering Workflow

The question-answering workflow begins when a user submits a natural language question within an existing conversation. The user interface forwards the request to the Laravel application together with the identifiers of the selected knowledge sources associated with the conversation.

The application first generates a vector embedding for the user's question using the embedding service. This query embedding is then compared with the stored document embeddings through semantic similarity search performed on the PostgreSQL vector database. Based on the similarity scores, the system retrieves the document chunks that are most relevant to the user's query.

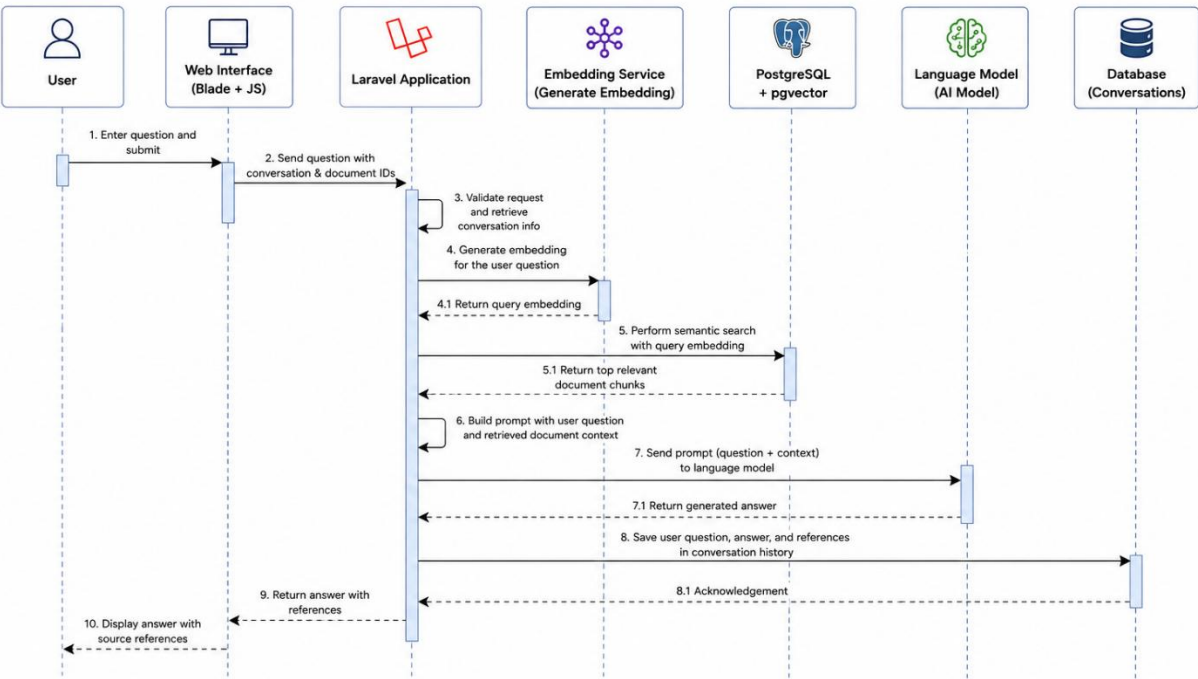


Figure 18: Sequence Diagram for AI-Assisted Question Answering Workflow

The retrieved document context, together with the user's original question, is subsequently forwarded to the language model. Rather than processing entire documents, the language model receives only the selected document fragments that are considered most relevant by the retrieval mechanism. This Retrieval-Augmented Generation approach improves response relevance while reducing unnecessary information processing.

After the language model generates a context-aware response, the Laravel application stores both the user's question and the generated answer within the conversation history. Any associated document references are also preserved to improve transparency and allow users to verify the origin of the retrieved information. Finally, the completed response is returned to the user interface, where it is displayed within the conversation together with the corresponding document references.

6.5. Activity Diagrams

Activity diagrams describe the flow of activities performed within the AI-powered Retrieval-Augmented Generation (RAG) system. While sequence diagrams focus on the interaction between system components over time, activity diagrams emphasize the logical flow of actions, decisions, and outcomes within a process.

Two activity diagrams were created to represent the most important workflows of the system. The first diagram illustrates the document upload and processing workflow, while the second diagram presents the AI-assisted question answering workflow.

Document Upload and Processing Activity

The document upload and processing activity begins when a user selects and submits a supported document through the web interface. The application validates the uploaded file based on predefined rules, such as file type and size. If the validation fails, the system displays an error message and stops the process.

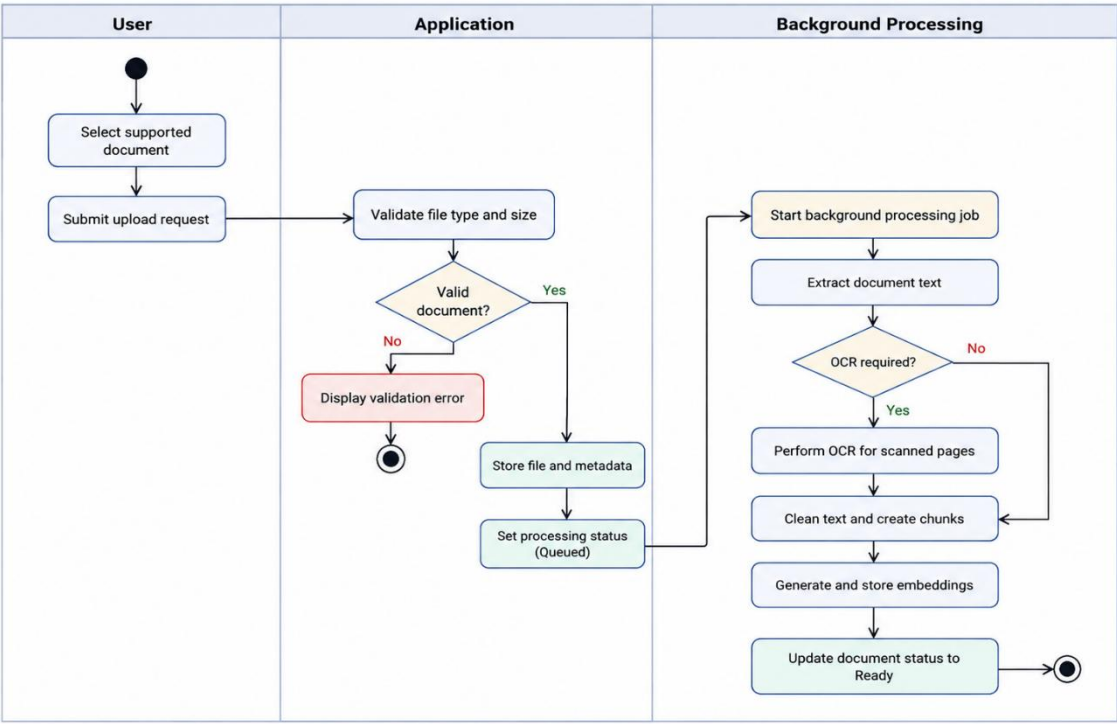


Figure 19: Activity Diagram for Document Upload and Processing

If the document is valid, the system stores the file and related metadata, updates the initial processing status, and starts the background processing workflow. The document processing stage includes text

extraction, optional OCR for scanned documents, text cleaning, chunk creation, embedding generation, and storage of processed content in the database.

After all processing steps are completed successfully, the document status is updated to indicate that it is ready for querying. This activity flow ensures that users can upload documents efficiently while long-running processing tasks are handled in the background without blocking the user interface.

Question Answering Activity

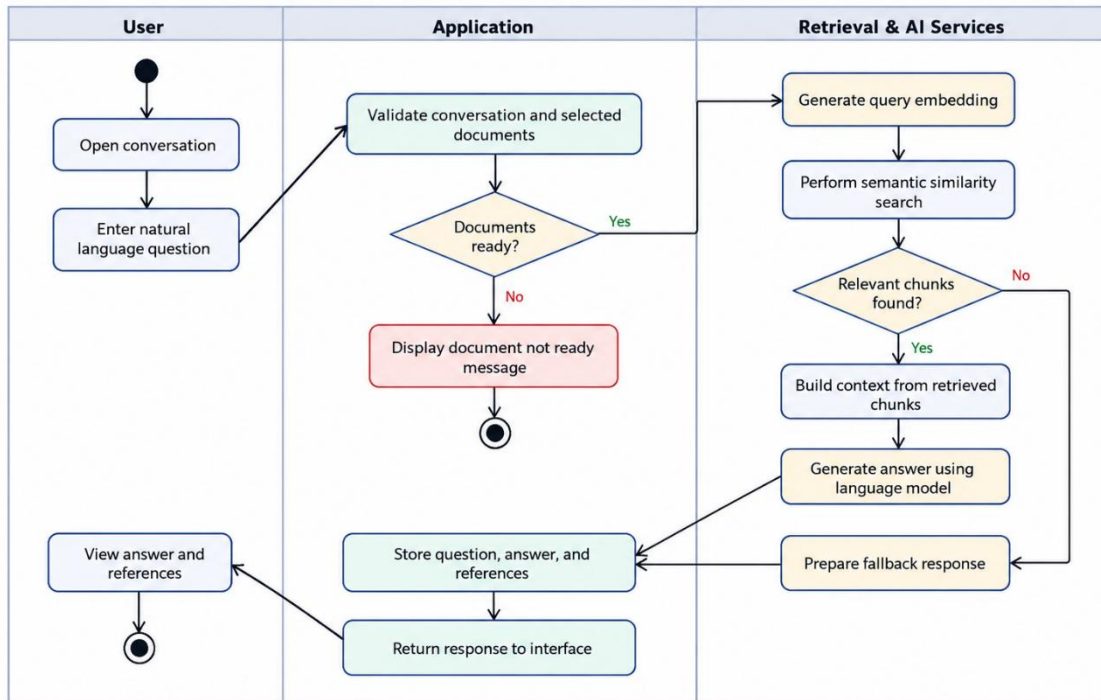


Figure 20: Activity Diagram for AI-Assisted Question Answering Workflow

The question answering activity begins when a user opens an existing conversation and submits a natural language question. The application first validates the conversation and checks whether the selected documents are already processed and ready for querying. If the documents are not ready, the system informs the user and stops the workflow.

If the selected documents are available, the system generates an embedding for the user question and performs semantic similarity search against the stored document chunks. If relevant chunks are found, they are used to build the contextual input for the language model. The language model then generates a response based on the retrieved document content.

The generated answer, the original question, and the related document references are stored in the conversation history. Finally, the response is returned to the interface and displayed to the user together with the relevant source references. If no relevant document content is found, the system prepares an appropriate fallback response instead of generating an unsupported answer.

6.6. Database Design

The database design of the AI-powered document retrieval system was developed to support secure document management, background document processing, semantic search, and conversation-based question answering. PostgreSQL serves as the primary relational database, while the pgvector extension provides persistent storage for both relational data and document embeddings required for semantic retrieval (**PostgreSQL Global Development Group, 2025; pgvector, 2025**). The database schema follows a normalized relational design that separates users, documents, document chunks, conversations, and chat messages into independent entities. This structure improves maintainability, minimizes data redundancy, and allows each component of the system to evolve independently. The overall database structure is illustrated in Figure 21.

Main Database Entities

The system consists of several core entities, each responsible for a specific aspect of the application's functionality.

User

The **users** table stores user account information required for authentication and authorization. Each registered user can upload multiple documents, create multiple conversations, and interact with the system independently. This relationship ensures complete isolation of user data while supporting multi-user operation.

Document

The documents table stores metadata for uploaded documents, including the document name, file location, and processing status throughout the ingestion pipeline. Each document belongs to a single user and can participate in multiple conversations after processing.

Document Chunk

After text extraction, every document is divided into smaller semantic chunks to improve retrieval accuracy. These chunks are stored in the **document_chunks** table together with their vector embeddings and metadata. Each chunk stores the extracted text together with its chunk order, page information, token count, embedding vector, and additional metadata. This table represents the knowledge base used during semantic retrieval. Similarity searches are performed directly on the embedding vectors using the pgvector extension.

Conversation

The **conversations** table stores every chat session initiated by a user. Each conversation represents an independent discussion and preserves the complete interaction history. A conversation can reference one or more uploaded documents through an intermediate relationship table, allowing users to ask questions across multiple documents simultaneously.

Conversation Message

The **conversation_messages** table stores every exchanged message within a conversation. Both user questions and AI-generated responses are preserved chronologically. Maintaining message history enables users to continue previous discussions without losing conversational context.

Conversation Document

The relationship between conversations and documents is maintained using the **conversation_documents** table. Instead of limiting a conversation to a single document, this design allows multiple documents to be attached to the same conversation. Consequently, the retrieval engine can search across several documents simultaneously before generating a response.

AI Usage Log

The **ai_usage_logs** table records interactions between the application and AI services, including usage metadata, processing status, token consumption, and references to related entities. It supports monitoring, troubleshooting, and future administrative reporting.

User Limits

The **user_limits** table stores configurable usage restrictions such as upload quotas, storage limits, and daily interaction limits. Keeping these settings separate from user accounts improves flexibility and simplifies future administrative enhancements.

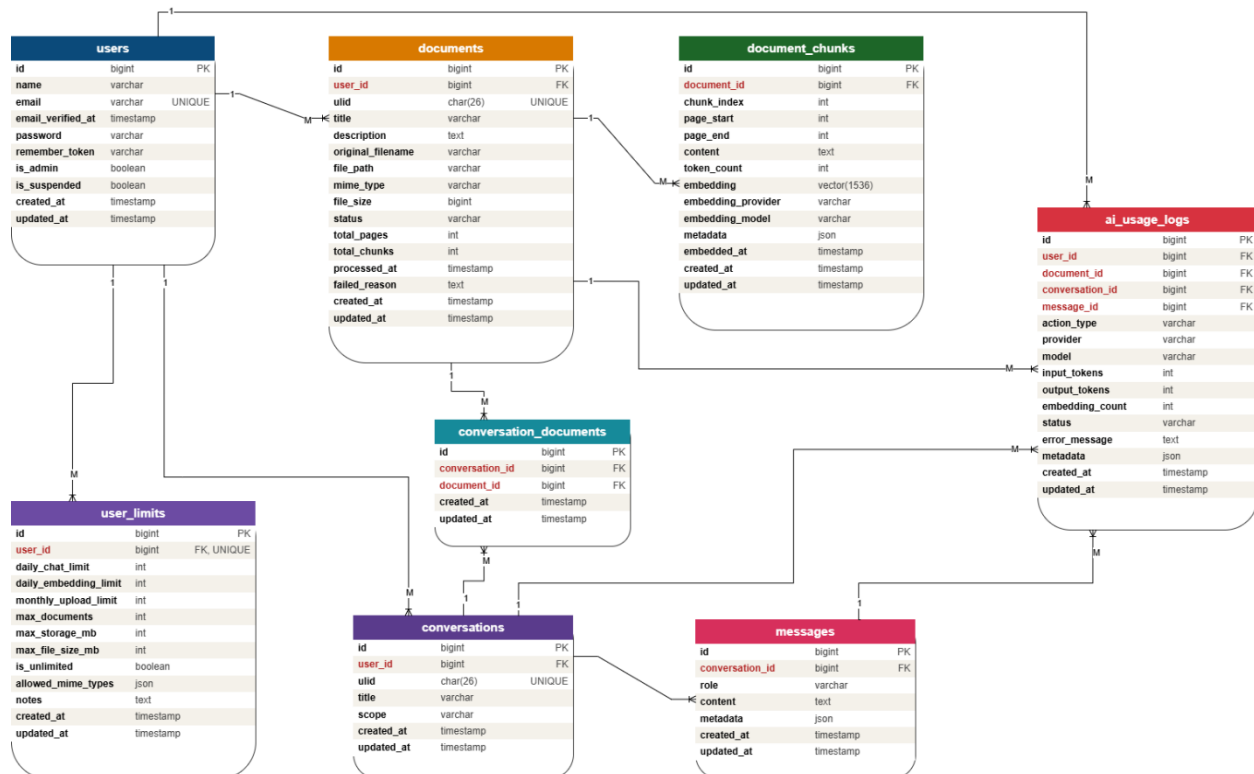


Figure 21: Entity Relationship Diagram of the AI-Powered Document Retrieval System

Entity Relationships

The relationships between the major entities are summarized below.

These relationships provide a flexible data model that supports document management, semantic indexing, and multi-document conversations while maintaining referential integrity throughout the database.

Parent Entity	Relationship	Child Entity
User	One-to-Many	Documents
User	One-to-Many	Conversations
User	One-to-One	User Limits
User	One-to-Many	AI Usage Logs
Document	One-to-Many	Document Chunks
Document	One-to-Many	AI Usage Logs
Conversation	One-to-Many	Messages
Conversation	Many-to-Many	Documents (via Conversation Documents)
Conversation	One-to-Many	AI Usage Logs
Message	One-to-Many	AI Usage Logs

Table 2: Relationships Between the Primary Database Entities

Overall, the database schema provides a scalable foundation for managing document processing, semantic indexing, and AI-assisted knowledge retrieval while remaining sufficiently modular for future enhancements.

7. Conclusion

The Software Specification and Design phase established a comprehensive foundation for the development of the AI-powered document retrieval system. Through the systematic analysis of user requirements, functional and non-functional specifications were identified and translated into a structured system design that addresses the project's objectives. The resulting specification provides a clear understanding of the system's expected behaviour while ensuring that both user needs and technical requirements are consistently reflected throughout the development process.

The design phase focused on defining the overall system architecture, major software components, database structure, user interface, and interaction workflows. By separating the system into well-defined modules, the proposed design promotes maintainability, scalability, and flexibility for future enhancements. The architectural, component, class, sequence, activity, and database diagrams collectively provide a clear representation of the system structure and serve as a valuable reference during implementation.

Overall, this Software Specification and Design report establishes a solid blueprint for implementing the AI-powered document retrieval system. The documented requirements, architectural decisions, and design artifacts provide a structured roadmap for development, reduce implementation risks, and support the consistent integration of future enhancements.

8. References

- Fowler, M. (2002). **Patterns of Enterprise Application Architecture**. Addison-Wesley. <https://martinfowler.com/books/ea.html>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley. <https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610>
- International Organization for Standardization. (2011). **ISO/IEC/IEEE 29148:2011 Systems and software engineering—Life cycle processes—Requirements engineering**. https://www.normservis.cz/download/view/iec/info_isoieee29148%7Bed1.0%7Den.pdf
- Laravel. (2025). **Laravel Documentation**. <https://laravel.com/docs>
- PostgreSQL Global Development Group. (2025). **PostgreSQL Documentation**. <https://www.postgresql.org/docs/>
- pgvector. (2025). **pgvector Documentation**. <https://github.com/pgvector/pgvector>
- Docker Inc. (2025). **Docker Documentation**. <https://docs.docker.com/>
- Tailwind Labs. (2025). **Tailwind CSS Documentation**. <https://tailwindcss.com/docs>
- Mozilla Developer Network. (2025). **JavaScript Guide**. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Tesseract OCR. (2025). **Tesseract OCR**. <https://tesseract-ocr.github.io/>
- Poppler Project. (2025). **Poppler**. <https://poppler.freedesktop.org/>
- Lewis, P., Perez, E., Piktus, A., et al. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. Advances in Neural Information Processing Systems, 33. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf